

Pomegranate Orchard Monitoring System

End-to-End Deep Learning for Smart Agriculture

Master's Project Report

<https://github.com/Samer-Gassouma/Pomegranate-Orchard-Monitor>

May 2026

1 Introduction

Pomegranate cultivation is a significant agricultural sector, yet manual quality inspection remains labor-intensive, subjective, and error-prone. This project presents an end-to-end deep learning system for automated pomegranate orchard monitoring that combines object detection, instance segmentation, and explainable artificial intelligence (XAI).

The system was designed as a two-stage pipeline: a YOLOv8m detector identifies and classifies individual fruits into six condition categories, while a U-Net ResNet-34 model segments the precise boundary of each detected fruit. A dual explainability layer provides human-interpretable justifications for model decisions, supported by quantitative validation metrics.

2 Dataset

The Makinay Pomegranate Dataset from Roboflow Universe was used for training and evaluation. It contains approximately 7,000 annotated images with instance segmentation labels (bounding boxes and polygon masks) across six classes:

Class ID	Class Name	Condition
0	crown_damaged	Defective
1	crown_good	Healthy
2	surface_damaged	Defective
3	surface_good	Healthy
4	surface_burnt	Defective
5	surface_cracked	Defective

Table 1: Pomegranate defect classification categories.

The dataset was downloaded in YOLOv8 format for detection training and COCO format for segmentation mask extraction. Training used a standard train/validation split with images resized to 640×640 for YOLO and 256×256 for U-Net.

3 Methodology

3.1 Stage 1: Object Detection with YOLOv8m

YOLOv8m (medium variant) was selected for its balance between detection accuracy and inference speed. The model was initialized from the Ultralytics COCO pre-trained checkpoint and fine-tuned on the pomegranate dataset for 100 epochs on an NVIDIA T4 GPU via Kaggle.

Training configuration:

- Image size: 640×640
- Batch size: 16
- Optimizer: SGD with momentum 0.937
- Learning rate: 0.01
- Augmentation: Mosaic, MixUp, random flip, HSV jitter

The model achieves strong in-distribution performance with mAP@0.5 of 0.987 and mAP@0.5:0.95 of 0.836 on the validation set.

3.2 Stage 2: Instance Segmentation with U-Net

A U-Net architecture with ResNet-34 encoder was implemented using the segmentation-models-pytorch library. The model outputs a single-channel binary mask (sigmoid activation) representing the fruit foreground boundary.

Training configuration:

- Encoder: ResNet-34 (ImageNet pre-trained)
- Loss: Binary Cross-Entropy + Dice Loss
- Optimizer: Adam with learning rate 10^{-4}
- Input: 256×256 cropped fruit regions
- Epochs: 50 with early stopping

It is important to note that the U-Net segments fruit boundaries rather than defect regions. This is honestly reported as fruit instance segmentation, not defect segmentation, to maintain scientific integrity.

3.3 Stage 3: Explainability

Two complementary explainability methods were integrated:

EigenCAM is applied to the YOLO backbone to visualize which image regions influenced detection decisions. It computes the principal eigenvector of the feature covariance matrix without requiring gradient computation, making it stable for detection architectures.

Grad-CAM is applied to the U-Net decoder to explain segmentation decisions. It uses gradient information flowing into the final convolutional layer to produce a coarse localization map highlighting important regions for the segmentation output.

3.4 Stage 4: Validation Metrics

Faithfulness measures whether the explanation identifies pixels the model actually relies upon. The top 20% most salient Grad-CAM pixels are masked, inference is re-run, and the confidence drop is measured. A high score indicates the CAM correctly identifies decision-critical pixels.

Cross-model alignment measures whether YOLO (detection) and U-Net (segmentation) attend to the same image regions. The IoU between EigenCAM and Grad-CAM attention regions is computed as an agreement score.

4 System Implementation

The system is deployed as a web application with three layers:

Backend: FastAPI provides REST endpoints for detection, segmentation, full analysis, and explainability. Models are loaded once during application startup via a lifespan context manager, ensuring efficient inference.

Frontend: Streamlit provides an interactive interface with image upload, confidence threshold and mask opacity sliders, an explanation toggle, side-by-side original and annotated display, per-fruit detailed analysis panels, and annotated image download.

Configuration: All parameters (model paths, thresholds, image sizes, class names, colors, XAI settings) are centralized in a single `config.yaml` file, enabling reproducible experiments without code modification.

Containerization: A Dockerfile enables single-command deployment:

```
docker build -t pom-monitor .  
docker run -p 8000:8000 pom-monitor
```

5 Results and Discussion

The YOLOv8m detector achieves excellent in-distribution performance with mAP@0.5 of 0.987 and mAP@0.5:0.95 of 0.836. The U-Net achieves high pixel accuracy on fruit boundary segmentation (Dice 1.0, IoU 1.0), though these values reflect the class-imbalanced nature of the task where fruit pixels dominate.

The system exhibits a known limitation of narrow-domain models: significant performance degradation on out-of-distribution images. When tested on pomegranate photos from different geographic regions or camera equipment, the model produces false-positive detections. This is documented honestly as a domain shift limitation rather than hidden.

6 Conclusion

This project successfully implements an end-to-end pomegranate orchard monitoring system integrating object detection, fruit instance segmentation, and dual explainability with quantitative validation. The system achieves strong in-distribution detection performance and provides interactive visualization through a web-based interface.

Future work includes expanding the dataset for better domain generalization, acquiring pixel-level defect annotations for true defect segmentation, and converting models to ONNX or TensorRT for edge deployment.

Technologies used: Python, PyTorch, Ultralytics YOLOv8, segmentation-models-pytorch, FastAPI, Streamlit, Docker, OpenCV, NumPy.

Training platform: Kaggle (NVIDIA T4 GPU).